AuctionHub

Multi-Type Auction Platform

Technical Specification

and API Contract

Course:	Software Systems Design (PSP)
Assignment:	Laboratory 1
Document version:	1.0
Authors:	*Klaidas Blandis, Arnas Vyšniauskas, Justinas Jarmalavičius, Deividas Matonis*

Contents

1 Product overview and scope **4**
1.1 Audience and business problem . . . 4
1.2 Objectives . . . 4
1.3 Included and excluded scope . . . 4
1.4 Normative language and document authority . . . 4
1.5 Assumptions . . . 4

2 Actors, terminology, and priorities **5**
2.1 MoSCoW interpretation . . . 5
2.2 Priority summary . . . 5

3 Functional requirements **6**
3.1 Baseline account policy . . . 6

4 Auction lifecycle and shared rules **8**
4.1 Time semantics . . . 8
4.2 Creation constraints . . . 8
4.3 Participant restrictions . . . 8

5 Auction algorithms **9**
5.1 Ascending-price auction . . . 9
5.2 Descending-price auction . . . 9
5.3 Sealed-bid auction . . . 9

6 Business flows and concurrency **10**
6.1 Publishing an auction . . . 10
6.2 Accepting a bid: sequence diagram . . . 10
6.3 Idempotency and transaction ordering . . . 10

7 Settlement, cancellation, and recovery **11**
7.1 Scheduled processing . . . 11
7.2 Settlement transaction . . . 11
7.3 Cancellation and blocking . . . 11
7.4 Recovery and reconciliation . . . 11

8 Notification and multi-device specification **12**
8.1 Channels and guarantees . . . 12
8.2 Persistence and delivery . . . 12

9 Realtime protocol and client synchronization **13**

9.1 Connection . . . 13
9.2 Delivery and reconnect behaviour . . . 13
9.3 Display correctness . . . 13

10 Non-functional requirements **14**
10.1 Reference verification environment . . . 14
10.2 Limits and retention . . . 14

11 High-level architecture **15**
11.1 Package diagram and dependency direction . . . 15
11.2 Suggested implementation stack . . . 15

12 Modules and class responsibilities **16**
12.1 Deployment and configuration . . . 16

13 Data model: accounts and auctions **17**

14 Data model: offers, results, and messaging **18**
14.1 Required indexes . . . 18

15 UI/UX wireframes and navigation **19**
15.1 Account, notifications, and moderation screens . . . 20
15.2 Screen transitions . . . 20

16 HTTP API conventions **21**
16.1 Authentication and authorization . . . 21
16.2 Pagination and query parameters . . . 21
16.3 Errors . . . 21

17 HTTP endpoint catalogue **22**

18 Request and response schemas **23**
18.1 Draft and publication . . . 23
18.2 Auction detail projection . . . 23
18.3 Offers and results . . . 23

19 Remaining interface details and security **25**
19.1 Account and inbox payloads . . . 25
19.2 Push endpoint validation . . . 25
19.3 Moderation payloads . . . 25
19.4 Privacy and authorization boundaries . . . 25
19.5 Operational observability . . . 25

20 Acceptance tests and traceability **26**

21 Implementation handover and design decisions **27**

21.1 Required implementation deliverables 27

21.2 Decisions fixed by this specification 27

21.3 Project risks and mitigations 27

21.4 Submission checklist 28

1 Product overview and scope

AuctionHub is a responsive web platform for selling individual items or indivisible bundles through different auction mechanisms. Sellers select a mechanism, publish the item and schedule, and let the platform handle bidding and settlement. Buyers can participate from multiple devices without relying on manual messages or locally running countdowns.

1.1 Audience and business problem

The core audience is individual sellers, small businesses, and buyers of goods such as collectibles, equipment, vehicles, and household items. These users need transparent rules, reliable deadlines, and a single authoritative auction result. Manual bidding through chat or social media creates ambiguity about the order of bids and whether an offer arrived on time. AuctionHub resolves these issues by validating all actions on the server and retaining their outcome.

1.2 Objectives

- Support ascending-price, descending-price, and sealed-bid auctions through one platform.
- Guarantee at most one winner for each auction, including under concurrent requests.
- Keep connected clients informed and restore correct state after reconnection.
- Retain personal notifications across devices, with optional browser push delivery.
- Provide a bounded implementation suitable for a student project.

1.3 Included and excluded scope

Included: local account registration and authentication; auction discovery; item images; draft creation; scheduling; bids or purchases; automatic settlement; watchlists; notifications; moderation; and an audit trail.

Excluded: payment collection, escrow, shipping, identity verification, taxes, refunds, reputation scores, chat, automatic proxy bidding, native mobile applications, multiple currencies, and automatic extensions of deadlines. An auction result records the platform's winner and winning amount; the system does not process the subsequent commercial transaction. Seller and winner exchange contact details through a restricted result view.

1.4 Normative language and document authority

Shall denotes a mandatory requirement. **Should** denotes a desirable feature. All monetary values, timing rules, and permissions in this document are normative. The accompanying OpenAPI file defines HTTP shapes; this document defines business semantics. Implementers shall report contradictions rather than silently choosing different rules. Priorities describe scope, not permission to change mandatory business rules.

1.5 Assumptions

The implementation serves one deployment and one currency (EUR). Each account represents one participant; coordinated bidding across different accounts is outside automated detection scope. The application uses HTTPS in deployed environments and a single PostgreSQL primary as the authority for auction time and ordering. Internet connectivity is required for participation.

2 Actors, terminology, and priorities

Actor	Permissions and responsibilities
Guest	Browse public auctions and public results; register or sign in.
Member	All guest operations; create and sell items; bid on other sellers' auctions; watch auctions; access own notifications and devices.
Administrator	All member operations; cancel scheduled or active auctions with a recorded reason; block accounts; inspect moderation history.
Scheduler	Internal process that starts and ends auctions and creates deadline reminders.
Delivery worker	Internal process that distributes persisted events and push notifications.

A seller and a bidder are contextual roles of a member, not separate account types. An administrator cannot bid on their own auctions and cannot alter accepted offers.

Term	Meaning
Ascending	Public increasing bids with a fixed minimum increment.
Descending	Public price decreases on a fixed schedule; first accepted purchase wins.
Sealed	One immutable hidden offer per member; highest offer wins at the deadline.
Accepted	Committed successfully to the database after validation.
Settlement	Atomic transition that determines the final result.
Notification	Durable personal record stored independently of delivery channels.
Realtime event	Ephemeral signal instructing a client to refresh authoritative state.
Push subscription	A browser installation's permission-dependent delivery endpoint.
Revision	Monotonically increasing auction version for committed auction mutations.

2.1 MoSCoW interpretation

Must requirements constitute the assessed implementation baseline. Should requirements may be omitted only if clearly identified in the implementation handover. Could requirements are extensions and shall not be prerequisites of Must flows. Won't requirements are explicitly excluded.

2.2 Priority summary

Priority	Scope
Must	Three auction types; account access; safe bidding and settlement; durable multi-device notifications; realtime refresh; optional-per-device push infrastructure; moderation; core responsive screens.
Should	Image previews, email verification, and administrator filtering by cancellation reason.
Could	Saved searches, notification email digests, additional item categories.
Won't	Payments, shipping, native apps, chat, proxy bids, reserve prices, bidding extensions.

Browser push infrastructure is mandatory; a user granting permission and an operating system actually displaying a push are not guaranteed.

3 Functional requirements

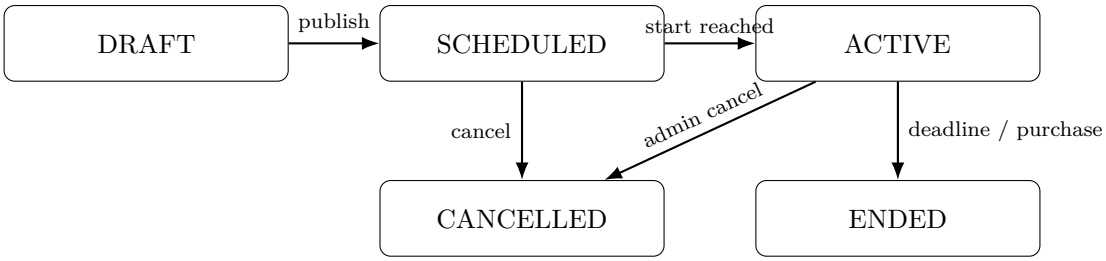
ID	Priority	Requirement
FR-01	Must	A guest shall register using unique email, display name, and password, and sign in or sign out.
FR-02	Must	A user shall view and revoke their active sessions. Blocking a user shall revoke all sessions.
FR-03	Must	Anyone shall browse and filter auctions by text, category, type, and state; results shall be paginated.
FR-04	Must	A member shall create and edit a draft containing the item, type, schedule, and applicable price parameters.
FR-05	Must	A seller shall attach zero to five validated item images to a draft.
FR-06	Must	Publishing shall validate all fields and make the auction publicly visible in SCHEDULED state.
FR-07	Must	The scheduler and command handlers shall apply time-dependent state transitions using database time.
FR-08	Must	The platform shall accept ascending bids only when they meet the current minimum.
FR-09	Must	A descending auction shall expose its current server-calculated price and accept at most one purchase.
FR-10	Must	A sealed auction shall accept at most one immutable offer per member and hide offers from others.
FR-11	Must	The platform shall settle each auction exactly once, recording the winner or absence of a winner.
FR-12	Must	Members shall watch and unwatch public, non-final auctions; this shall not place an offer.
FR-13	Must	Connected clients shall receive invalidation events and refresh changed auction data.
FR-14	Must	A member shall receive durable personal notifications for events defined in Section 8.
FR-15	Must	Reading a notification on one device shall update its state for all devices.
FR-16	Must	A member shall register, replace, or delete push subscriptions separately for each browser installation.
FR-17	Must	Notification history shall be available after disconnects regardless of push permission.
FR-18	Must	Sellers shall view their auctions; bidders shall view auctions in which they participated.
FR-19	Must	Seller and winner shall see each other's display name and email only after a successful settlement.
FR-20	Must	A seller shall delete a draft or cancel a scheduled auction before its start.
FR-21	Must	An administrator shall cancel a scheduled or active auction with a reason and block or unblock users.
FR-22	Must	The system shall record accepted offers, results, cancellations, and moderation actions for audit.
FR-23	Must	Repeating an offer request with the same idempotency key shall not create another offer or purchase.
FR-24	Must	Each mutation shall enforce ownership, role, and blocked-account restrictions on the server.
FR-25	Should	Registration should support email verification before auction participation. This extension requires a separate implementation decision.
FR-26	Could	Members could save search criteria for later reuse.

3.1 Baseline account policy

The Must baseline permits participation immediately after registration; no email-verification service is required. Password reset is excluded from this laboratory baseline. Administrators are provisioned

through a deployment command, never through public registration. Accounts cannot be deleted in the baseline because accepted offers and audit relationships must remain stable.

4 Auction lifecycle and shared rules



Deleting a draft removes it; it is not a lifecycle state. ENDED and CANCELLED are final. An ended auction has exactly one result with outcome SOLD or UNSOLD. A cancelled auction has no result and all previous offers remain in the audit history but cannot win.

4.1 Time semantics

All timestamps are UTC instants serialized as RFC 3339 strings ending in Z. UI timestamps are displayed in the device's timezone with an explicit timezone label. A scheduled auction starts when $t \geq startAt$ and permits participation while $startAt \leq t < endAt$. At $t = endAt$, no offer or purchase is valid.

For each command, the server locks the auction row and then reads PostgreSQL wall-clock time. That instant is the command's decision time. It is not the HTTP arrival time, client clock, transaction start timestamp, or eventual commit time. Any due transition is applied before command validation. A request waiting for a lock may therefore miss the deadline. Accepted offers preceding the deadline remain valid even if their transaction commits just after it.

4.2 Creation constraints

Title: 5–120 characters. Description: 20–5,000 characters of plain text. Category: one fixed value from COLLECTIBLES, VEHICLES, ELECTRONICS, HOME, EQUIPMENT, OTHER. Start: at least 5 minutes and at most 90 days after publication decision time. Duration: 10 minutes to 30 days. All prices are positive integer euro cents, maximum 100,000,000 cents (EUR 1,000,000). No floating-point currency calculations are permitted.

Drafts may be incomplete. Full validation occurs on publication. After publication, item fields, type, schedule, images, and price parameters are immutable. The seller may cancel only while still scheduled under the server's time check. There is no seller cancellation of an active auction.

4.3 Participant restrictions

The seller cannot participate in their auction. A blocked account cannot authenticate, create or change data, or maintain an event connection. Blocking does not invalidate previously accepted offers: the blocked user can still win. Administrators may separately cancel a live auction when necessary. Existing published auctions of a blocked seller continue unless explicitly cancelled. These rules avoid silently changing results through account moderation.

5 Auction algorithms

5.1 Ascending-price auction

Configuration contains `startingPriceCents` and `incrementCents`, both positive. With no accepted bids, the minimum valid bid equals the starting price. Otherwise it equals the highest accepted amount plus the increment. Overbidding by more than one increment is allowed. An existing highest bidder may bid again. Equal amounts and retractions are rejected.

The server locks the auction, applies due transitions, checks eligibility, and compares the submitted amount against the current minimum. It appends an offer, updates the highest-offer reference, increases the revision, and creates relevant outbox events in one transaction. If the next minimum would exceed the global price maximum, further bids are rejected with PRICE_LIMIT. At settlement, the highest accepted offer wins and pays its offered amount. There is no hidden reserve price and no deadline extension.

5.2 Descending-price auction

Configuration contains `startingPriceCents`, `floorPriceCents`, `decrementCents`, and `stepSeconds`. Starting price shall exceed the floor; decrement shall be positive; step shall be an integer from 10 to 86,400 seconds. The floor must be reached strictly before the deadline:

$$\left\lceil \frac{startingPriceCents - floorPriceCents}{decrementCents} \right\rceil \cdot stepSeconds < endAt - startAt.$$

For active time t, the authoritative price is

$$P(t) = \max\left(floorPriceCents,\ startingPriceCents - \left\lfloor \frac{t - startAt}{stepSeconds} \right\rfloor \cdot decrementCents\right).$$

The floor remains available until the deadline. Price steps do not create bids or database writes. Clients calculate an indicative countdown from server time and refresh at price boundaries.

A purchase request contains `expectedPriceCents`. Under the lock, the server computes $P(t)$ and accepts only if it exactly matches the expected amount. A changed price causes PRICE_CHANGED and returns the new price for explicit reconfirmation. A successful purchase atomically creates a PURCHASE offer and SOLD result and transitions directly to ENDED. Concurrent losers receive AUCTION_NOT_ACTIVE. If nobody purchases, settlement at the deadline records UNSOLD.

5.3 Sealed-bid auction

Configuration contains `minimumOfferCents`. Each member may submit one amount at or above that minimum. The offer cannot be edited, withdrawn, or replaced. A unique database constraint enforces one offer per bidder per sealed auction.

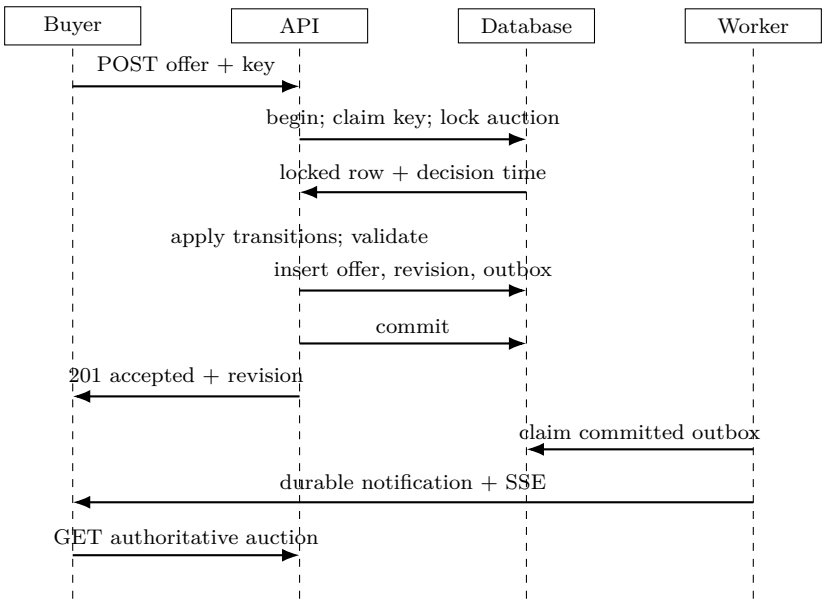
Before and after settlement, other participants and the seller cannot retrieve individual sealed offers, identities, or the offer count. Each bidder can retrieve their own offer. Administrators receive moderation metadata, not hidden offer amounts through normal APIs. At settlement the server selects the greatest amount; ties are resolved by the lowest per-auction offer sequence. The winner pays their own offered amount. Public results reveal the winning amount but not the winner's identity. No offers means UNSOLD.

6 Business flows and concurrency

6.1 Publishing an auction

Seller creates a draft, completes its fields, uploads images, and selects Publish. The server locks the draft, verifies ownership and completeness, and rechecks the schedule against current database time. On success it changes the state to SCHEDULED, increments revision, and commits. The seller sees the scheduled detail screen. Validation errors identify fields and preserve the draft. A second publish request on the same auction returns INVALID_STATE and does not duplicate it.

6.2 Accepting a bid: sequence diagram



6.3 Idempotency and transaction ordering

Offer and purchase commands require a UUID `Idempotency-Key`. The key is scoped to the account and retained for 7 days. The server stores method, route, canonical body hash, HTTP status, and response body. A same-key/same-request retry returns the stored response. A different request with that key returns IDEMPOTENCY_CONFLICT. The account and key are claimed atomically before locking the auction; parallel duplicates wait for the original transaction and replay its response. Rolled-back internal failures do not persist a success. Stable business rejections are retained; authentication failures and rate-limit responses are not.

A lost response shall be resolved by retrying the identical command with the same key. The UI must not report a timeout as a definite rejection or create a fresh key automatically. Auction locks serialize bids, purchases, settlement, and cancellation. Transactions touching multiple auctions lock them in ascending UUID order. Outbox publication happens only after commit, so failed transactions never announce successful bids.

7 Settlement, cancellation, and recovery

7.1 Scheduled processing

The scheduler polls due auctions at least once per second using indexed timestamps and row locks with skip-locked processing. It starts SCHEDULED auctions at their start and settles ACTIVE auctions at their deadline. Multiple workers may run safely because state is rechecked while holding the row lock. If downtime spans both boundaries, the worker transitions through ACTIVE and settles in the same transaction without accepting retrospective offers.

Read endpoints return an effective state derived from timestamps even if the persisted transition is delayed: a scheduled row past its start is presented as ACTIVE; a row past its deadline awaiting settlement is presented as FINALIZING. FINALIZING is a response-only state that disables participation and indicates that the result is pending. Mutating commands always execute the due transition transactionally. Once settled, reads return ENDED and the stored result. List filters use effective states as well.

7.2 Settlement transaction

1. Lock the auction and sample decision time; verify that it is due or a valid descending purchase is being accepted.
2. If already final, return the stored outcome without inserting another result.
3. Determine the winner using the type-specific rules; insert one result under a unique auction constraint.
4. Set ENDED and endedAt, increment revision, and append AUCTION_ENDED to the outbox.
5. Commit; notification materialization and channel delivery follow asynchronously.

The unique result constraint is the final protection against duplicate outcomes. A retry after worker failure must produce the same stored result. An external message broker is not a source of truth.

7.3 Cancellation and blocking

Seller cancellation is allowed only before start. Administrator cancellation is allowed in SCHEDULED or ACTIVE, requires a 10–500 character reason, records the actor and time, and generates notifications. Cancellation and a concurrent bid/purchase are ordered by the auction lock: if purchase settles first, cancellation fails; if cancellation commits first, the bid/purchase fails. Ended auctions cannot be cancelled or rewritten.

Blocking a member locks the account, marks it blocked, revokes sessions and push subscriptions, and records an audit action in one transaction. Participation commands hold a shared account lock before acquiring the auction lock; blocking takes an exclusive account lock. A command that wins the account lock first can finish before the block, while later commands fail. Unblocking does not restore sessions or push registrations. The user signs in again.

7.4 Recovery and reconciliation

After restart, the scheduler processes all overdue auctions. The outbox worker retries unprocessed records. Notification inserts have a unique recipient/event/type key. A reconciliation job every minute detects overdue unsettled auctions and unprocessed outbox records older than 60 seconds, logs an alert, and requeues eligible work. Audit records and accepted offers are append-only; operational repair must never silently change a winner.

8 Notification and multi-device specification

8.1 Channels and guarantees

Durable inbox: the authoritative notification record belongs to the account, not a device. It is retained for 180 days. **Server-sent events (SSE):** one authenticated stream per open tab signals auction or inbox changes. **Web Push:** an optional delivery channel per browser installation using a service worker and permission granted by the user. The platform does not guarantee operating-system display, delivery to offline devices, or push support on every browser. Every supported browser can still use the inbox.

Event	Recipients	Timing / deduplication
OUTBID	Previous highest ascending bidder, unless the new bidder is the same account	One per accepted replacing offer
STARTING_SOON	Current watchers of a scheduled auction	Once, at start minus 5 min
ENDING_SOON	Current watchers and distinct accepted bidders; descending watchers only	Once, at end minus 5 min, if non-final
WON	Winner	Once per SOLD result
ENDED	Watchers and distinct bidders, excluding seller and winner	Once per result
SELLER_RESULT	Seller	Once per result; SOLD or UNSOLD
CANCELLED	Seller, watchers, and distinct bidders, excluding cancelling actor	Once per cancellation

Recipients are evaluated when the logical event is materialized, using records created no later than the event's timestamp; deleted watch rows are excluded. Reminders are emitted only when their threshold is crossed, while the auction remains eligible. After downtime, a start reminder is skipped if the start has passed and an end reminder is skipped if the end has passed. Users joining a watchlist after a reminder has already been generated do not receive it retroactively. Simultaneous recipient roles are deduplicated. Blocked recipients are excluded from new personal notifications.

8.2 Persistence and delivery

The auction transaction creates a domain outbox event. A worker atomically materializes recipient notifications and per-subscription delivery jobs, then marks the event processed. Large fan-out may use batches with a persisted cursor; the event is complete only after all recipients are processed. Unique keys make replay safe. A removed watch row before materialization may suppress a reminder; notification delivery is not a deadline guarantee for bidding.

Each notification has an opaque ID, account ID, type, auction ID, plain-text title/body, createdAt, and nullable readAt. Only the owner may retrieve or mark it read. A repeated read action preserves the first readAt. Reading emits an inbox invalidation to all account streams; it does not delete notifications from other devices. There is no per-device unread counter.

Push payloads contain notification ID, generic title, and an application-relative auction URL. They exclude sealed amounts, contact information, and credentials. Clicking opens the auction; private content requires sign-in. Push jobs retry after 5 seconds, 30 seconds, 2 minutes, and 10 minutes, then fail without removing the inbox record. Expired notifications (24 hours) are not pushed. HTTP 404/410 disables a subscription; 429 honours Retry-After, bounded by expiry. Repeated push display is possible, so clients use notification ID as the display tag.

9 Realtime protocol and client synchronization

9.1 Connection

`GET /api/v1/events` returns `text/event-stream` using the same secure session cookie as REST. A tab sends `auctionIds` as a comma-separated list of at most ten public auction UUIDs. Own drafts may be subscribed to; other users' drafts are denied. Notifications are always scoped to the authenticated user. Guests poll public details every 5 seconds instead. The server closes streams when sessions expire or are revoked and checks authorization at least every 30 seconds.

```
event: auction.updated
data: {"auctionId":"uuid","revision":18,"reason":"OFFER_ACCEPTED"}

event: inbox.updated
data: {"reason":"CREATED"}

event: sync.required
data: {"reason":"CONNECTED"}

: heartbeat
```

Auction event reasons are STARTED, OFFER_ACCEPTED, PURCHASED, ENDED, or CANCELLED. Inbox reasons are CREATED or READ. Events do not include bid amounts or private identities. Event names and JSON fields above are the complete baseline contract. Clients ignore unknown event types and additional fields.

9.2 Delivery and reconnect behaviour

SSE is an invalidation channel with best-effort delivery; it has no historical replay or event ID contract. On every connection the server sends `sync.required`; the client fetches current subscribed auction details, own offers, and the inbox. Duplicate and out-of-order events are allowed. Clients ignore auction revisions no greater than their loaded revision; if a greater revision arrives during a fetch, they refetch after it completes.

Heartbeat comments are sent every 15 seconds. If the client sees no stream activity for 45 seconds, it closes the stream and reconnects after 1, 2, 4, 8, 16, then 30 seconds, with up to 20% random jitter. The server may close slow clients with more than 100 buffered messages; they recover through a new snapshot. At most five streams per account are allowed; excess connections receive 429 before streaming begins.

9.3 Display correctness

Responses contain `serverTime`; auction details include `nextPriceChangeAt` for descending auctions, or null at the floor. The client estimates server offset using the request midpoint and recalibrates on every refresh. Countdown and local price animations are indicative. Server validation remains authoritative.

An open auction page refetches every 15 seconds even when SSE appears healthy, at scheduled start/end boundaries, and at descending price boundaries. During disconnect it displays "Live updates disconnected; reconnecting" and retains the last refresh time. Forms may still submit over working HTTP, but success is shown only after confirmation. On tab focus, network recovery, or push click, the client refreshes immediately. A received event never directly declares a winner; only a fetched committed result does.

10 Non-functional requirements

ID	Measurable requirement	Verification
NFR-01	At most one result and one winner shall exist per auction under 100 concurrent purchase/settlement requests.	Concurrency test
NFR-02	With 200 signed-in users, 200 SSE streams and 20 mutation requests/second, 95% of REST responses shall complete within 500 ms, excluding image transfer.	10-min load test
NFR-03	Under the same load and healthy network, 95% of auction invalidations shall arrive within 2 s of commit; inbox records within 5 s.	Timestamp sampling
NFR-04	Healthy workers shall persist due starts and settlements within 3 s of their deadline for 95% of auctions; commands still enforce exact cutoffs.	Scheduler test
NFR-05	State shall survive application and worker restarts with no loss of committed offers or results.	Restart test
NFR-06	All deployed traffic shall use TLS; session cookies shall be Secure, HttpOnly, SameSite=Lax. Passwords shall be stored as Argon2id hashes using a maintained library.	Config review
NFR-07	State-changing endpoints shall reject missing/invalid CSRF tokens; output shall escape item text. Queries shall be parameterized.	Security tests
NFR-08	Core screens shall work at 360 px and 1440 px width without horizontal page scrolling; keyboard users shall reach every action and see focus.	UI inspection
NFR-09	Errors shall include stable machine codes and human-readable messages; logs shall exclude passwords, cookies, push keys, and sealed amounts.	Test / log review
NFR-10	A nightly database backup shall be restorable; target recovery-point loss is at most 24 h and recovery time at most 4 h.	Restore exercise
NFR-11	The deployed service should achieve 99% monthly availability, excluding declared maintenance. This is a target, not a guaranteed SLA.	Uptime monitor
NFR-12	All Must acceptance scenarios shall pass from a documented clean deployment.	Handover run

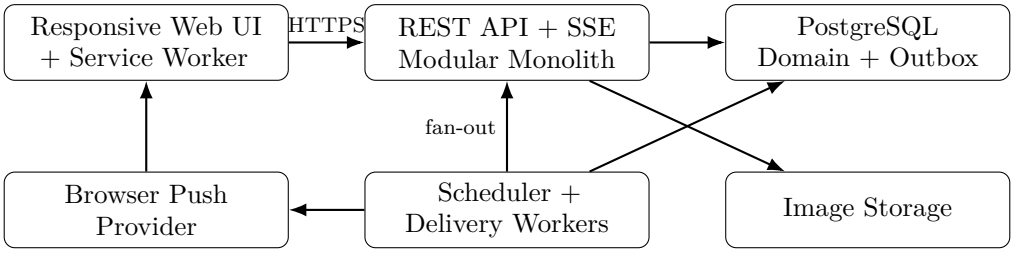
10.1 Reference verification environment

Performance targets assume an application host with 4 vCPU/8 GB RAM, PostgreSQL with 2 vCPU/4 GB RAM and SSD storage, and at most 50 ms client round-trip latency. Dataset: 10,000 auctions, 100,000 offers, and 100,000 notifications. The implementation report shall state actual hardware, test data, duration, and measured percentiles. External push latency is excluded from NFR-03.

10.2 Limits and retention

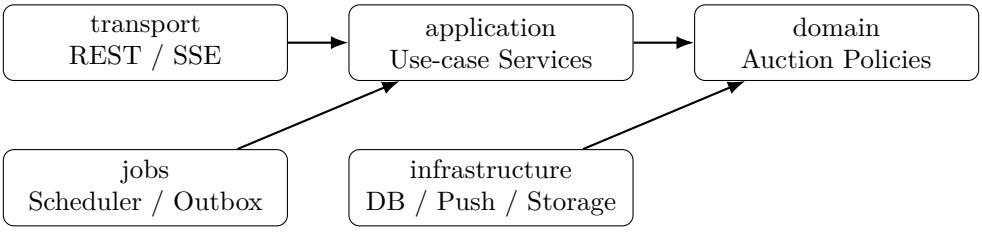
JSON requests are limited to 64 KiB. Auction search text is at most 100 characters. Per account: 60 mutations/minute and 20 offer/purchase attempts per 10 seconds, enforced across tabs. Login: 10 attempts/minute per IP and email pair; public reads: 120/minute per IP. 429 includes Retry-After seconds. Limits may be configured but their deployed values shall be documented. Accepted offers, results, and audit records are retained throughout the laboratory deployment; notification and idempotency retention are as specified above.

11 High-level architecture



The backend is a modular monolith with separate worker processes sharing one database. One API process is sufficient for the baseline. If multiple API replicas are deployed, a shared pub/sub adapter shall broadcast invalidations to every replica; notifications and results remain durable in PostgreSQL. Redis pub/sub is a permitted optional adapter, not required for single-replica deployment. Workers shall not rely on process-local queues for durable work.

11.1 Package diagram and dependency direction



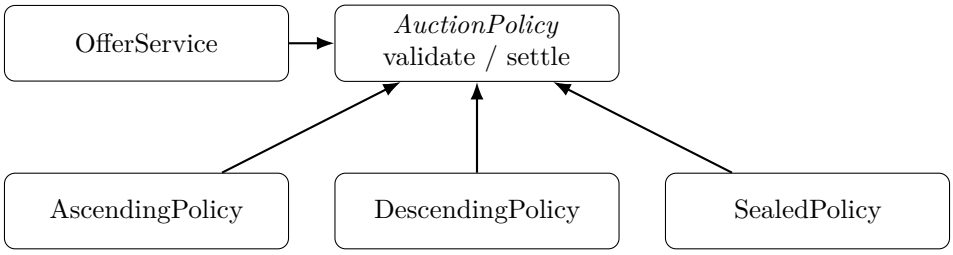
Application services depend on repository, transaction, clock, and channel interfaces defined inside the backend. Infrastructure implements those interfaces. Domain auction policies contain no HTTP, storage, or UI code. Frontend code depends only on the published API contract.

11.2 Suggested implementation stack

TypeScript frontend and backend, a component-based SPA, PostgreSQL, a maintained HTTP framework, and a Web Push library are suggested. Equivalent technologies are allowed if the observable contracts and transactional rules are preserved. Build scripts and dependency versions shall be pinned by the implementing team. No cloud-specific service is required.

12 Modules and class responsibilities

Module / class	Responsibility
IdentityService	Registration, password verification, sessions, CSRF, blocked-account guards.
AuctionService	Draft editing, publication, watchlists, ownership and lifecycle validation.
OfferService	Idempotent command orchestration, account/auction locks, offer persistence.
AuctionPolicy	Type-specific validation and result selection behind one interface.
AscendingPolicy	Minimum next bid and greatest-offer selection.
DescendingPolicy	Time-derived price and expected-price validation.
SealedPolicy	Single-offer constraint and amount/sequence result ordering.
SettlementService	Idempotent due transitions and atomic result creation.
NotificationService	Recipient selection, inbox creation, read synchronization.
OutboxWorker	Durable event claiming, retries, recipient batching.
PushAdapter	Subscription validation, payload sending, provider error classification.
ModerationService	Cancellation, blocking, audit recording.
AuctionQueryService	Public projection, effective states, hidden-data filtering, pagination.



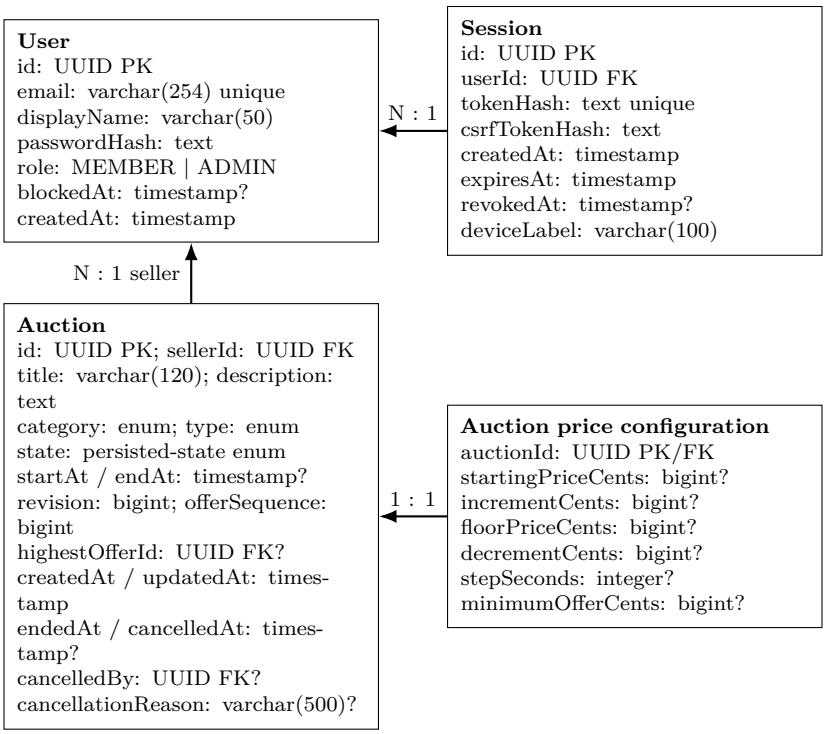
The diagram expresses interface implementation and usage, rather than a complete code-level class hierarchy. Data entities are specified separately below.

12.1 Deployment and configuration

Deliverables shall include a reproducible local composition for frontend/API, worker, database, and image volume. Required configuration: database URL, public origin, session secret, push VAPID public/private keys and contact identifier, storage location, and rate limits. Secrets belong in environment configuration, not source control. Migrations run before application startup. The health endpoint is live when the process responds; readiness additionally checks the database and migration version. Worker freshness and outbox age shall be exposed through metrics.

13 Data model: accounts and auctions

All IDs are UUIDs unless stated otherwise. Timestamp means UTC timestamp with time zone. Nullable fields are marked "?". Monetary fields are signed 64-bit integers constrained to the allowed positive range. Text lengths below are maxima. The diagrams show stored fields, not methods.

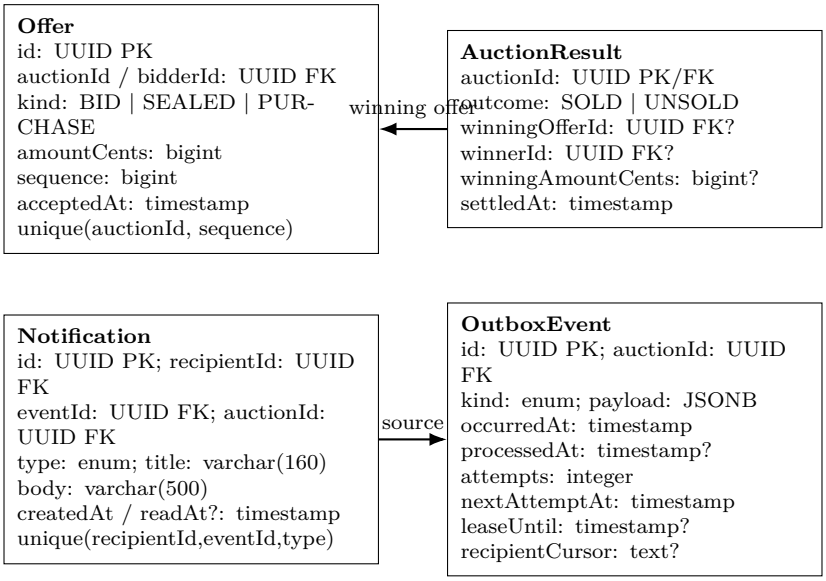


Auction type enum: ASCENDING, DESCENDING, SEALED. Persisted state enum: DRAFT, SCHEDULED, ACTIVE, ENDED, CANCELLED. Draft schedule/configuration fields may be null. Published auctions require exactly their type's fields, with all inapplicable price fields null. Check constraints enforce shape and positivity; publication validation enforces time-relative conditions.

Entity	Additional fields and constraints
AuctionImage	id PK, auctionId FK, storageKey text unique, position integer 0–4, mediaType enum JPEG/PNG/WEBP, sizeBytes integer, createdAt timestamp; unique (auctionId, position).
Watch	userId FK + auctionId FK composite PK, createdAt timestamp. No watch of a draft or own auction.
Session	Token stored only as a hash; absolute lifetime 7 days. Multiple sessions per account are supported.

Email is trimmed and lowercased before uniqueness checks. Display names are 2–50 characters and need not be unique. Passwords are 12–128 characters. Public auction projections never expose password hashes, session details, seller email, or bidder account IDs.

14 Data model: offers, results, and messaging



Entity	Fields / integrity rules
PushSubscription	id UUID PK, userId FK, endpoint text unique, p256dh text, auth text, deviceLabel varchar(100), createdAt timestamp, disabledAt timestamp?. Endpoint/key material is private.
PushDelivery	id UUID PK, notificationId FK, subscriptionId FK, state PENDING/SENT/FAILED, attempts integer, nextAttemptAt timestamp, expiresAt timestamp, lastErrorCode varchar(100)?; unique(notificationId,subscriptionId).
IdempotencyRecord	userId FK + key UUID composite PK; method varchar(10), path text, requestHash text, responseStatus integer, responseBody JSONB, createdAt/expiresAt timestamp.
AuditAction	id UUID PK, actorId FK?, auctionId FK?, targetUserId FK?, action enum, reason varchar(500)?, createdAt timestamp; immutable.
ReminderMarker	auctionId FK + kind STARTING_SOON/ENDING_SOON composite PK, eventId FK unique. Prevents duplicate logical reminders.

SOLD requires all winning fields; UNSOLD requires them all null. Winning offer shall belong to the same auction and winner. Use a composite foreign key or equivalent transactional invariant. Partial unique indexes enforce one SEALED offer per (auctionId,bidderId) and one PURCHASE per auction. Every accepted offer increments the auction's sequence while locked. Foreign keys restrict deletion of referenced business records; only draft-owned images/configuration cascade on draft deletion.

14.1 Required indexes

Auction(state,startAt), Auction(state,endAt), Auction(sellerId,createdAt,id), Auction(category,type,createdAt,id); Offer(auctionId,amountCents DESC,sequence ASC), Offer(bidderId,auctionId); Watch(auctionId,createdAt); Notification(recipientId,createdAt DESC,id DESC), Notification(recipientId,readAt); OutboxEvent(processedAt,ne PushDelivery(state,nextAttemptAt); Session(userId). Search may use a database text index; no external search engine is required.

15 UI/UX wireframes and navigation

The wireframes are low fidelity. Brackets denote controls, underlines denote inputs, and lists denote panels. All private screens require sign-in. Layout collapses to one column on narrow screens; feedback is textual as well as colour-based.

```
AUCTIONHUB [Browse] [Sign in]
Search: [_________________] [Search]
Category [All v] Type [All v]
State [Active v]
[Image] Item title | Type
Current price / Sealed offers
Ends at ... [Open]
[Previous] Page 1 [Next]
```

```
SIGN IN / REGISTER
Email [_________________]
Password [______________]
[Sign in] [Create account]
Registration adds display name
and password confirmation.
Errors appear below fields.
Return to requested screen.
```

```
AUCTION DETAIL
Title | Category | Seller name
[Images] Description
Type | Status | Server countdown
Current / minimum price
Amount [________] [Place bid]
or [Buy at displayed price]
or [Submit sealed offer]
[Watch / Unwatch]
Public bid history (ascending)
Connection status / last refresh
Final result / cancellation reason
```

```
CREATE / EDIT DRAFT
Title [_________________]
Description [___________]
Category [v] Type [v]
Start [____] End [____]
Type-specific price inputs
[Upload images] [Remove]
[Save draft] [Preview]
[Publish] [Delete draft]
Validation next to each field
Publish confirmation: immutable
rules and scheduled times
```

An active ascending detail page shows the minimum next bid and anonymous offer amounts/times; it never promises acceptance before the server responds. A sealed detail page replaces public bid history with “Your offer” or “No offer submitted” and warns that submission is final. A descending purchase confirmation displays the exact EUR amount; PRICE_CHANGED requires renewed confirmation. DRAFT and SCHEDULED show their state and disable participation. FINALIZING shows “Result pending” and refreshes until settled.

15.1 Account, notifications, and moderation screens

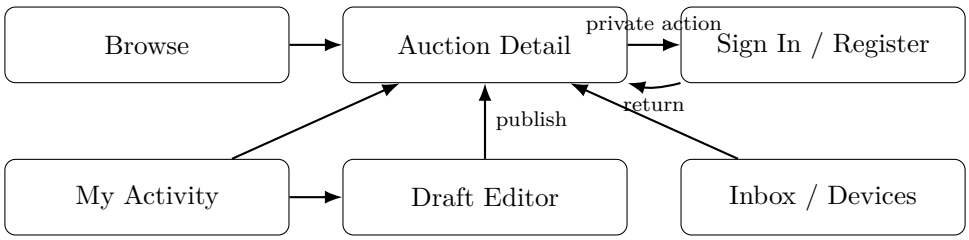
```
MY ACTIVITY
[Selling] [Participating] [Watching]
Item | State | My offer / Result
[Open] [Edit draft]
[Cancel scheduled]
[Create auction]
RESULT (seller / winner only)
Winning price | Counterparty
Display name | Contact email
```

```
NOTIFICATIONS [Unread:  3]
[Unread only] [All]
New:  You have been outbid
Item title | timestamp
[Open auction] [Mark read]
Read:  Auction ended ...
[Next page]
Inbox works without push.
```

```
ACCOUNT / DEVICES
Display name | Email
This browser:  push disabled
[Enable push] / [Disable push]
Other push registrations
Label | [Remove]
Active sign-in sessions
Label | Created | [Revoke]
[Sign out]
```

```
ADMINISTRATION
[Auctions] [Users] [Audit]
Search [____________]
Auction | State | [Open]
[Cancel auction] -> Reason field
User | Block status
[Block] / [Unblock] -> Reason
Action history / timestamp
```

15.2 Screen transitions



The persistent navigation exposes Browse, My Activity, Inbox (unread count), Account, and New Auction; administrators also see Administration. Opening a notification marks it read and navigates to its auction. Cancel/delete/block actions require a confirmation dialog. Buttons are disabled during submission; timeout feedback offers retry with the same idempotency key. Push permission is requested only after clicking Enable push. Denial leaves inbox access unchanged.

16 HTTP API conventions

Base path: `/api/v1`. JSON uses camelCase. IDs are opaque UUID strings; cents are JSON integers. Unknown request fields are rejected with VALIDATION_ERROR. Responses may add fields without breaking clients. All successful JSON responses include `serverTime`. Lists contain `items`, `nextCursor` (null if none), and `serverTime`. Default page size is 20; allowed range is 1–100.

16.1 Authentication and authorization

Authentication uses an opaque 256-bit random session cookie named `auctionhub_session`, Path=/, lifetime 7 days. `GET /auth/session` returns the current user and CSRF token for a valid session. The login/register response also returns that token. All authenticated mutations require `X-CSRF-Token`; registration/login require an allowed Origin and JSON content type. Same-origin deployment is the baseline; cross-origin credentials are disabled. Session tokens are never returned in JSON.

Unauthenticated private requests return 401. Blocked authenticated requests return 403 ACCOUNT_BLOCKED. Forbidden actions return 403; inaccessible or nonexistent drafts return 404 to avoid leaking their existence. Unauthorized sealed-offer list access returns 403. Admin checks are applied on every administration request.

16.2 Pagination and query parameters

Opaque base64url cursors encode the last (createdAt,id) pair and active filters and are validated on reuse. Default ordering is createdAt descending, then id descending. New inserts may appear only after restarting pagination; this is not a frozen snapshot. Auction history uses sequence descending. Invalid or filter-mismatched cursors return 400 INVALID_CURSOR. Auction filters: `q`, `category`, `type`, `state`; personal activity additionally uses `scope=selling|participating|watching`. Public lists exclude DRAFT; own selling lists may include it. Default state filter includes SCHEDULED, ACTIVE, FINALIZING; explicit filters may include final states.

16.3 Errors

```
{"error":{"code":"BID_TOO_LOW","message":"Minimum bid is 12500 cents.",
 "details":{"minimumBidCents":12500},"requestId":"uuid"},
 "serverTime":"2026-11-03T12:00:00Z"}
```

Status	Codes
400	VALIDATION_ERROR, INVALID_CURSOR, MISSING_IDEMPOTENCY_KEY
401 / 403	UNAUTHENTICATED / FORBIDDEN, ACCOUNT_BLOCKED, CSRF_INVALID
404 / 413	NOT_FOUND / PAYLOAD_TOO_LARGE
409	INVALID_STATE, AUCTION_NOT_ACTIVE, BID_TOO_LOW, PRICE_CHANGED, OFFER_EXISTS, IDEMPOTENCY_CONFLICT, EMAIL_EXISTS, PRICE_LIMIT
415 / 429	UNSUPPORTED_MEDIA_TYPE / RATE_LIMITED
500 / 503	INTERNAL_ERROR / SERVICE_UNAVAILABLE

Validation details contain `fields`, a map of field names to messages. PRICE_CHANGED includes `currentPriceCents`; BID_TOO_LOW includes `minimumBidCents`. Unexpected errors return no internal stack trace.

17 HTTP endpoint catalogue

All paths below are relative to `/api/v1`. “Member” includes administrators; owner means the relevant account or seller.

Method	Path	Access	Success behaviour
POST	/auth/register	Guest	201 user, csrfToken; sets cookie
POST	/auth/login	Guest	200 user, csrfToken; sets cookie
GET	/auth/session	Member	200 user, csrfToken
DELETE	/auth/session	Member	204 revoke current session
GET	/sessions	Member	200 own sessions list
DELETE	/sessions/{id}	Owner	204 revoke; idempotent
GET	/auctions	Public	200 filtered public list
GET	/me/auctions	Member	200 scope-filtered personal list
POST	/auctions	Member	201 draft
GET	/auctions/{id}	Public/owner	200 detail projection
PATCH	/auctions/{id}	Seller	200 updated draft
DELETE	/auctions/{id}	Seller	204 delete draft
POST	/auctions/{id}/publish	Seller	200 scheduled detail
POST	/auctions/{id}/cancel	Seller/admin	200 cancelled detail; reason required
POST	/auctions/{id}/images	Seller	201 image; multipart file
DELETE	/auctions/{id}/images/{imageId}	Seller	204 delete draft image
POST	/auctions/{id}/offers	Member	201 accepted ascending/sealed offer
POST	/auctions/{id}/purchase	Member	201 accepted purchase and result
GET	/auctions/{id}/offers	Public	200 ascending history only
GET	/auctions/{id}/my-offer	Member	200 own latest offer, or null
GET	/auctions/{id}/result	Public	200 public result; null until settled
GET	/auctions/{id}/contact	Seller/winner	200 counterpart; SOLD only
PUT	/auctions/{id}/watch	Member	204 add watch; idempotent
DELETE	/auctions/{id}/watch	Member	204 remove watch; idempotent
GET	/notifications	Member	200 own list; optional unreadOnly
PUT	/notifications/{id}/read	Owner	200 updated notification
GET	/notifications/unread-count	Member	200 count
GET	/push/public-key	Member	200 publicKey
GET	/push/subscriptions	Member	200 own device metadata
POST	/push/subscriptions	Member	201 subscription or 200 existing
DELETE	/push/subscriptions/{id}	Owner	204 remove; idempotent
GET	/events	Member	200 SSE stream
GET	/admin/users	Admin	200 users; q/blocked filters
PUT	/admin/users/{id}/block	Admin	200 blocked user; reason required
DELETE	/admin/users/{id}/block	Admin	200 unblocked user
GET	/admin/audit	Admin	200 audit list; auctionId/userId filters
GET	/health/live, /health/ready	Public	200 status, or 503 not ready

The server shall reject blocking oneself or another administrator with FORBIDDEN. Repeated block/unblock operations return current state without duplicating audit records. Repeated deletion of an already missing own subscription/session returns 204; IDs belonging to others return 404. Draft/image deletion returns 404 when already removed. Cancelling a final auction returns INVALID_STATE. Watch creation is allowed only in SCHEDULED/ACTIVE; removal remains allowed after the auction ends.

18 Request and response schemas

18.1 Draft and publication

```
POST /auctions
{"title":"Vintage camera","description":"Working camera with lens and case.",
 "category":"COLLECTIBLES","type":"ASCENDING",
 "startAt":"2026-12-01T12:00:00Z","endAt":"2026-12-01T13:00:00Z",
 "pricing":{"startingPriceCents":10000,"incrementCents":500}}
// DESCENDING pricing:
{"startingPriceCents":20000,"floorPriceCents":10000,
 "decrementCents":1000,"stepSeconds":60}
// SEALED pricing:
{"minimumOfferCents":10000}
```

POST requires only type; other draft fields are optional. PATCH accepts any subset except type, which is immutable even in draft. Supplied pricing replaces the complete pricing object; null values clear draft fields. Title/description/category/schedule/pricing may be cleared in draft. Publishing uses an empty JSON object. Cancellation uses `\{"reason":"Item no longer available"\}`. Images use multipart field `file`; at most 5 MiB each, JPEG/PNG/WebP only, decoded dimension at most 4096 pixels per side. Server verifies decoded content, strips metadata, generates safe storage names, and rejects SVG/HTML. Draft images are accessible only to the owner through authenticated storage routes; published images are public. No upload can fetch a remote URL.

18.2 Auction detail projection

```
{"id":"uuid","seller":{"id":"uuid","displayName":"Seller"},
 "title":"Vintage camera","description":"...","category":"COLLECTIBLES",
 "type":"ASCENDING","state":"ACTIVE","revision":4,
 "startAt":"2026-12-01T12:00:00Z","endAt":"2026-12-01T13:00:00Z",
 "createdAt":"2026-11-30T10:00:00Z","pricing":{
 "startingPriceCents":10000,"incrementCents":500},
 "currentPriceCents":11000,"minimumBidCents":11500,
 "nextPriceChangeAt":null,"images":[],"isWatching":false,
 "cancellationReason":null,"result":null,
 "serverTime":"2026-12-01T12:10:00Z"}
```

For ascending auctions currentPrice is the highest bid or starting price. For descending it is the effective price, and minimumBid is null. For sealed both are null. Outside the active interval, descending currentPrice is null; starting/floor prices remain visible in pricing. Sealed pricing reveals only its minimum. `isWatching` is false for guests. Images contain id, url, position. Public lists use the same projection without description, pricing, cancellationReason, or result. Public seller IDs identify sellers only; bidder IDs never appear in history or results.

18.3 Offers and results

```
POST /auctions/{id}/offers
Idempotency-Key: 4d70d8fa-458f-4e82-91c8-c5295ae8d315
{"amountCents":11500}
// Response 201 (own accepted offer)
{"offer":{"id":"uuid","auctionId":"uuid","kind":"BID",
 "amountCents":11500,"sequence":3,"acceptedAt":"...Z"},
 "revision":5,"serverTime":"...Z"}
POST /auctions/{id}/purchase
{"expectedPriceCents":18000}
// Response 201: offer, revision, serverTime, plus result:
{"outcome":"SOLD","winningAmountCents":18000,"settledAt":"...Z"}
```

Public ascending history contains id, amountCents, sequence, acceptedAt; no bidder identity. My-offer returns the latest own offer for ascending and the sole own offer for other types. Result responses wrap `result`, nullable until settlement. SOLD contains outcome, winningAmountCents, settledAt; UNSOLD has null winningAmountCents. Only contact responses disclose counterpart displayName and email. Before a SOLD result, contact access returns INVALID_STATE; a non-party receives FORBIDDEN.

19 Remaining interface details and security

19.1 Account and inbox payloads

Registration: email, displayName, password. Login: email, password. Authentication success contains user (id, email, displayName, role), csrfToken, serverTime. Session list records contain id, deviceLabel, createdAt, expiresAt, isCurrent. No endpoint returns stored token hashes. The server generates device labels from a sanitized user-agent summary; users need not supply them.

Notification list records contain id, type, auctionId, title, body, createdAt, readAt. Unread-count returns count and serverTime. Mark-read uses an empty object. Push registration accepts endpoint, keys (p256dh, auth), and deviceLabel. It returns only id, deviceLabel, createdAt, serverTime; listing includes disabledAt. Re-registering the same endpoint for the same user updates keys and re-enables it. Ownership transfer to another account requires deleting the browser's old subscription and generating a fresh one; otherwise the server returns FORBIDDEN.

19.2 Push endpoint validation

Only HTTPS endpoints belonging to an administrator-configured allowlist of push-provider hostnames are accepted. The application does not send requests to arbitrary user-provided hosts, private addresses, loopback, or redirected destinations. The adapter rejects redirects and validates resolved destination addresses at delivery time. Endpoint length is at most 2,048 characters. Key material must be valid base64url in the expected Web Push format. This prevents the push registration API from acting as a general server-side HTTP proxy.

19.3 Moderation payloads

Block and cancellation use a reason string of 10–500 characters. Admin user records expose id, email, displayName, role, blockedAt, createdAt. Audit records expose id, actorId, auctionId, targetUserId, action, reason, createdAt. Actions are PUBLISHED, OFFER_ACCEPTED, SETTLED, CANCELLED, USER_BLOCKED, USER_UNBLOCKED. Audit records must not expose sealed amounts; an offer reference may be recorded instead. Unblock takes no body.

19.4 Privacy and authorization boundaries

Contact details are available only to the seller and winning account after SOLD. Public results have no participant identities. Sealed offers are filtered at query construction, not merely hidden in the browser. Push payloads are intentionally generic because devices may show them on locked screens. A sign-out deletes only the current session and closes its streams; push subscriptions are account registrations and remain until explicitly removed. The UI shall offer "Disable push on this browser" adjacent to sign-out for shared devices.

19.5 Operational observability

Structured logs include requestId, route, status, duration, and safe domain IDs. Metrics include request error rates, lock wait duration, auction transition delay, oldest pending outbox age, notification materialization delay, push failure counts, and open stream count. Failed outbox events retry with exponential backoff capped at 5 minutes and trigger an operational alert after 10 failures; they are retained for manual requeue. Health responses must not contain secrets or full connection strings.

20 Acceptance tests and traceability

These scenarios are implementation acceptance criteria, not claims that the currently supplied design has been implemented or tested.

Test	Given / when / expected outcome	Requirements
AT-01	Register a new email; duplicate normalized email returns EMAIL_EXISTS. Sign-in sets a secure session; sign-out makes subsequent private reads return 401.	FR-01–02
AT-02	Publish incomplete draft: field errors and DRAFT retained. Publish valid draft: SCHEDULED. Any subsequent editing fails. Another user cannot retrieve a draft.	FR-04–06,24
AT-03	Ascending starts at 10000 with increment 500. First 10000 succeeds, 10499 then fails, 10500 succeeds. Seller bid fails.	FR-08,24
AT-04	Two concurrent ascending bids of 10500 after 10000: exactly one accepted; other sees new minimum. No equal highest bids exist.	FR-08; NFR-01
AT-05	At decision time endAt minus 1 ms a valid offer can succeed; at endAt it fails. Client clock changes do not affect either outcome.	FR-07–11
AT-06	Descending 20000, floor 10000, decrement 1000, 60-s steps: at 59.999 s price 20000; at 60 s 19000; after floor reached it stays 10000.	FR-09
AT-07	Submit purchase expecting old price after a step: PRICE_CHANGED, no purchase. Submit 100 simultaneous correct-price purchases: one SOLD result and one accepted purchase.	FR-09,11
AT-08	Two sealed offers tie; lower server sequence wins. Second offer by the same user fails. Seller and other bidders cannot fetch the offers or count, before or after settlement.	FR-10–11,24
AT-09	Deadline without accepted offers yields UNSOLD for all three types. Re-running settlement does not add a result or duplicate notifications.	FR-11,14
AT-10	Lose the accepted offer HTTP response; retry same key/body: identical offer ID and response. Same key/different amount: IDEMPOTENCY_CONFLICT.	FR-23
AT-11	Outbid user has two devices: one inbox record appears on both, push jobs target both registrations; marking read on one updates the other after refresh.	FR-14–17
AT-12	Disconnect one device, finish auction, reconnect: current result and notifications are fetched without SSE replay. Denied push permission does not hide inbox history.	FR-13–17
AT-13	Crash worker after notification insert but before acknowledgement; retry produces one notification per recipient/event/type. Expired push endpoint is disabled.	FR-14–17
AT-14	Race cancellation against a purchase: serialized result is either CANCELLED with no result, or ENDED/SOLD with cancellation rejected.	FR-11,20–22
AT-15	Block an account: new commands fail, sessions are revoked, accepted offers remain eligible. Blocking self/another administrator fails.	FR-02,21,24
AT-16	Non-party contact request fails; seller and winner can retrieve counterpart after SOLD. Public responses contain no email or bidder identity.	FR-19,24
AT-17	Restart after both start/end deadlines: settle overdue auction once, skip stale reminders, deliver durable pending result notifications.	FR-07,11,14
AT-18	Apply load, restart, backup-restore, keyboard, narrow-screen and security checks with the environment defined in NFRs. Report measured results.	NFR-02–12
AT-19	Search/filter/page auctions, watch/unwatch, upload valid and invalid images, and navigate personal activity without hidden or duplicated mutations.	FR-03,05,12,18

21 Implementation handover and design decisions

21.1 Required implementation deliverables

1. Source repository with frontend, backend, worker, migration, and infrastructure directories.
2. Reproducible setup instructions, environment example without secrets, migration and administrator-provisioning commands.
3. An executable API matching the accompanying OpenAPI contract and documented SSE semantics.
4. Seed data with at least one scheduled, active, sold, unsold, and cancelled auction, covering all types; users for seller, bidder, and administrator roles.
5. Automated domain/concurrency tests and a written acceptance report mapping AT IDs to outcomes.
6. A list of omitted Should/Could features and any deviations approved during implementation.

21.2 Decisions fixed by this specification

Decision	Rationale / consequence
Three auction mechanisms	Directly addresses the general auction-platform scope while sharing one lifecycle.
Single-item auctions, EUR	Keeps allocation and currency semantics precise and implementable.
Database time and row locks	Defines deadline and race outcomes without relying on client clocks.
Fixed ascending deadline	Avoids ambiguous extension rules and reminder rescheduling.
One immutable sealed offer	Makes hidden-offer ownership, ties, and revision rules explicit.
Descending expected-price check	Prevents a purchase at a different displayed amount without confirmation.
Durable inbox plus optional push	Provides cross-device history even when push display is unavailable.
SSE invalidation plus refetch	Keeps private data out of broadcasts and provides a clear reconnect protocol.
Transactional outbox	Allows notification recovery after process failure without losing committed events.
No payment processing	Focuses the assignment on auction coordination and reliable notifications.

21.3 Project risks and mitigations

Race conditions: isolate transactional commands early and run AT-04/07/14 before completing UI polish. **Notification loss:** implement durable inbox and outbox before adding push. **Scope growth:** retain the explicit exclusions and avoid adding auction types mid-implementation. **Unclear UI outcomes:** distinguish pending requests, rejected requests, and committed results. **Unsupported push environment:** demonstrate push on a documented supported device and demonstrate inbox recovery on every test browser.

21.4 Submission checklist

Replace administrative cover placeholders, confirm team and lecturer details, compile this source, and visually inspect all pages and diagrams. Submit the shared document/PDF together with `openapi.yaml` and retain a final snapshot at the deadline. The API contract is an attachment to this specification; neither the source nor the design asserts that a working system already exists.